

9. JavaScript – FetchAPI, WebSocket

Asynchronní komunikace

- JavaScript je jednovláknový a běží v Event Loopu, aby se tedy požadavky neblokovaly používají se asynchronní Web API prohlížeče
- Po dokončení async požadavku se vrací jako Promises nebo Callback Queue
- Základem všech požadavků jsou Promises, je to objekt reprezentující výsledek async operace (Stavy: pending, fulfilled, rejected)
- Async a Await pro označení asynchronního kódu

FetchAPI

- Nahradilo to zastaralé XMLHttpRequest
- Pro získávání dat přes HTTP protokol
- Model request-response
- Vysoká reže (každý dotaz má hlavičku)
- Nehází error při klasických chybách, nutnost *response.ok*
- CORS – bezpečnostní mechanismus, omezení HTTP požadavků na doméně z cizích zdrojů, pokud to server explicitně nedovolí

GET požadavek:

```
const response = await fetch('https://api.priklad.cz/data');
if (response.ok) { // Kritické: kontrola HTTP statusu 200-299
  const data = await response.json(); // Převedení streamu na JSON
}
```

POST požadavek:

```
const response = await fetch(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ jmeno: 'Jan' })
});
```

WebSocket

- Trvalé full-duplex spojení
- Server může klientovy posílat data bez toho, aby se zeptal
- Keep-alive spojení (ping/pong rámce)
- Nízká režie, komunikace binárními nebo textovými rámci
- HTTP hlavička *Upgrade: websocket* a server odpovídá *101 Switching Protocols*
- Cyklus: *new WebSocket('ws://...')* -> *onopen* -> *onmessage* -> *send()* -> *onclose*
- Chatovací apps, akciové grafy, online hry